

Phase 9 - phase9_report

- [PHASE 9 — MULTI-AGENT DECISION INTELLIGENCE](#)
 - [Real Estate AI Decision Intelligence Platform — Phase Completion Report](#)
 - [1. PHASE STATUS](#)
 - [2. OBJECTIVE](#)
 - [3. SECTION 9.2/9.1 COMPLIANCE — REUSE, NOT REIMPLEMENTATION](#)
 - [4. WHAT WAS BUILT](#)
 - [5. THE DECISION SCORE, DOCUMENTED](#)
 - [6. TWO REAL BUGS FOUND AND FIXED DURING BUILD](#)
 - [6B. POST-DELIVERY AUDIT FIX — inherited `roi\_total\_holding\_period` bug \(now fixed\)](#)
 - [7. DEMO BATCH RESULTS \(n=26: 25 from Phase 6's demo sample + property 1, included specifically to demonstrate the INSUFFICIENT path\)](#)
 - [8. WHY THE EXPLANATION AGENT IS A TEMPLATE, NOT AN LLM CALL](#)
 - [9. TESTS EXECUTED / PASSED / FAILED](#)
 - [10. MASTER PROMPT REQUIREMENTS COVERAGE](#)
 - [11. BUSINESS VALUE](#)
 - [12. KNOWN LIMITATIONS \(see also `docs/limitations.md`\)](#)
 - [13. DEPENDENCIES](#)
 - [14. SECURITY CONSIDERATIONS](#)
 - [15. PERFORMANCE CONSIDERATIONS](#)
 - [16. PRODUCTION READINESS](#)
 - [17. NEXT PHASE DEPENDENCIES](#)
 - [18. FINAL STATUS](#)

PHASE 9 — MULTI-AGENT DECISION INTELLIGENCE

Real Estate AI Decision Intelligence Platform — Phase Completion Report

Follows the Master A-Z Prompt's 10-phase architecture, Section 19. This is the integration phase: Phases 4-8 are wired into one controlled decision workflow. Matches the established convention:

`src/<topic>/*.py` modules, orchestrators, numbered CSVs, charts, automated tests.

1. PHASE STATUS

PHASE 9 — COMPLETE. 11 agents, a deterministic Decision Policy, and a full end-to-end orchestrator are built and wired together. **37/37 automated tests passing** (36 original + 1 audit-fix regression test - see Section 6B), including 8 true end-to-end tests that run the entire pipeline. All four decision states (INVEST, HOLD, AVOID, INSUFFICIENT) are confirmed reachable with real properties from the dataset.

2. OBJECTIVE

Integrate the intelligence from Phases 4-8 into a controlled decision workflow (§9.1) that answers “*Should I invest in Property X?*” — where the multi-agent system orchestrates, and a deterministic policy decides. No agent, and no LLM, invents the final decision.

3. SECTION 9.2/9.1 COMPLIANCE — REUSE, NOT REIMPLEMENTATION

Every agent that touches Phase 4-8 math calls that phase's own module directly:

Agent	Calls (unchanged from)
Market Agent	<code>data_loader._market_intelligence()</code> + <code>location_score._market_strength_score()</code> — Phase 8
Prediction Agent	<code>data_loader.build_property_master()</code> + <code>governance.one_hot()</code> + the trained <code>.joblib</code> artifacts — Phase 4
Evidence Agent	<code>HybridRetriever</code> + <code>rerank()</code> — Phase 5
Finance Agent	<code>engine.run_financial_engine()</code> + <code>scenarios.run_scenarios()</code> — Phase 6
Risk Agent	<code>monte_carlo.run_monte_carlo()</code> + <code>risk_metrics.compute_risk_metrics()</code> + <code>decision_stability.compute_decision_stability()</code> — Phase 7
Geo Agent	<code>geo_engine.run_geo_engine()</code> — Phase 8
Evidence Validator	<code>grounding.validate_answer()</code> — Phase 5

No agent recomputes NOI, IRR, a location score, or a retrieval ranking. The Decision Policy (§9.4) is the only place new arithmetic happens in this phase, and it operates only on the five already-computed component scores — never on raw property data.

4. WHAT WAS BUILT

11 agents (`src/agents/`): Query Understanding, Data, Market, Prediction, Evidence, Finance, Risk, Geo, Evidence Validator, Decision Policy, Explanation — each returning a standard `AgentFinding` contract (§9.3: `agent_name`, `status`, `finding_type`, `summary`, `metrics`, `evidence_ids`, `source`, `confidence`, `limitations`, `timestamp`).

decision_policy.py (§9.4-9.8): the deterministic core — `compute_component_scores()` — Market/Prediction/Financial/Risk/Geo, each 0-100 - `compute_decision_score()` — weighted sum, renormalizes when a component is missing - `compute_decision_confidence()` — **kept separate from score**, six independent completeness checks - `check_critical_gates()` — property found, financial data available, evidence not blocked, minimum confidence met - `decide()` — deterministic INVEST/HOLD/AVOID/INSUFFICIENT

orchestrator.py (§9.9): the only place that calls every agent in sequence and assembles the final `DecisionResult` (§9.8, all 19 required fields present).

explanation_agent.py: a deterministic template — explicitly *not* an LLM call in this phase (see Section 8 below for why).

5. THE DECISION SCORE, DOCUMENTED

Component	Weight	Computed from
Market score	20%	Market Agent's <code>market_strength_score</code> (reused from Phase 8)
Prediction score	15%	$50 + \text{valuation_gap_pct} \times 2.5$, capped [0,100] — undervalued vs. Model A's prediction scores higher
Financial score	25%	$50 + (\text{IRR} - \text{discount_rate}) \times 500$, capped [0,100]
Risk score	20%	$0.6 \times (100 - \text{prob_negative_NPV} \times 100) + 0.4 \times$

		stability_pct
Geo score	20%	Geo Agent's location_score (reused from Phase 8)

Every linear mapping above is a **documented judgement call**, not fitted to any historical investment-outcome dataset (none exists in this platform) — stated plainly in `decision_policy.py`'s docstrings, the same honesty standard every prior phase's thresholds have used.

Decision confidence is a *separate* number: the mean of six status-derived completeness checks (data, model, evidence, financial, risk, geo). Critically, **risk_completeness is hard-capped at 60/100 regardless of how good the simulated numbers look**, because Phase 7's `calibration_status` is permanently `UNCALIBRATED` (§7.7) — a dedicated test (`test_risk_completeness_capped_regardless_of_status_text`) locks this in.

Decision thresholds: INVEST requires score ≥ 65 **and** confidence ≥ 55 ; AVOID is score ≤ 35 ; everything else is HOLD — including a high score (≥ 65) that fails the confidence bar, which downgrades to HOLD rather than INVEST (verified by `docs/decision_charts/score_vs_confidence.png` — every demo-batch point above the INVEST threshold also clears the confidence bar, and the one point that doesn't clear confidence is INSUFFICIENT for an unrelated gate reason, not a near-miss INVEST).

6. TWO REAL BUGS FOUND AND FIXED DURING BUILD

Bug 1 — module name collisions. Phases 6, 7, and 8 each independently named their data-loading module `data_loader.py`. In isolation (one phase's tests) this was fine. Combined in Phase 9's single process, Python's module cache silently returned whichever `data_loader.py` loaded *first* to every subsequent `from data_loader import ...`, regardless of which phase's agent was asking. **Fixed** by renaming Phase 8's copy to `geo_data_loader.py` and Phase 4's copy to `predict_data_loader.py` (filename-only changes, zero logic modified, verified by diff) and updating the handful of internal references within each subpackage.

Bug 2 — Evidence Agent citation cross-contamination. The Evidence Agent's first version joined multiple evidence snippets as `"{content} [{evidence_id}]"` separated by spaces. Phase 5's `grounding.split_sentences()` splits text right after `[.!?]\s+`, so a citation marker sitting *after* one document's period and *before* the next document's first word got parsed as belonging to the **next** sentence, not the one it was meant to cite. This caused the Evidence Validator to correctly flag several properties' evidence as `BLOCKED` (numbers attributed to the wrong citation) — the validator caught a real defect exactly as designed. **Fixed** by re-terminating each block as `"{content_without_trailing_period} [{evidence_id}]."`, keeping the citation marker inside its own sentence's punctuation. A regression test (`test_evidence_validator_no_cross_document_contamination`) checks this across 5 properties/localities.

6B. POST-DELIVERY AUDIT FIX — inherited `roi_total_holding_period` bug (now fixed)

An independent review (the same review that found this same inherited-state-copy pattern in Phase 7) found that `src/finance/engine.py` here was pulled into Phase 9 *before* Phase 6's own post-delivery audit fixed `roi_total_holding_period` (it was computing a gross cash multiple instead of a net return, overstating ROI by exactly 100 percentage points in every case). Decision Policy's `financial_score` does **not** use this field (it uses IRR directly — see Section 5), so no decision, score, or confidence value in this phase was affected. However, `FinanceAgent.metrics["roi_total_holding_period"]` (surfaced to any downstream consumer, e.g. Phase 10, that reads the full `financial_findings` payload) was carrying the wrong value.

Fixed by re-copying Phase 6's corrected `engine.py` (re-verified byte-identical via `diff`), with one new regression test (`test_finance_agent_roi_is_net_return_not_gross_multiple`) added directly to this phase's own suite so it independently re-checks the fix from the Finance Agent's own interface, not just by trusting Phase 6/7's tests — **suite is now 37/37 passing**. `docs/decision_results_csv/*.csv` were regenerated; the decision distribution (INVEST 6 / HOLD 12 / AVOID 7 / INSUFFICIENT 1) came back byte-

for-byte identical, confirming no decision-relevant output was ever affected by this bug.

7. DEMO BATCH RESULTS (n=26: 25 from Phase 6’s demo sample + property 1, included specifically to demonstrate the INSUFFICIENT path)

Decision	Count
INVEST	6
HOLD	12
AVOID	7
INSUFFICIENT	1 (property 1 — no expense/rental history, confirmed back in Phase 6)

Decision score: min 21.8, median 56.2, max 69.2. Decision confidence: min 49.2 (the INSUFFICIENT case), median 75.8, max 81.7 — confidence never reaches the “fully confident” range even at its best, because `risk_completeness` is permanently capped at 60 by design. This is intentional and correctly reported, not a bug: **the platform is telling the truth about the ceiling its own uncalibrated risk model imposes on every decision’s confidence.**

8. WHY THE EXPLANATION AGENT IS A TEMPLATE, NOT AN LLM CALL

Stated explicitly in `explanation_agent.py`’s docstring: Phase 10 (\$10.10) is where the platform’s actual LLM “AI Explanation Layer” belongs. Building Phase 9’s Explanation Agent as a deterministic Python template rather than an LLM call keeps (a) this phase’s own test suite fully deterministic — `test_e2e_deterministic_reproducibility` checks the exact same decision comes out twice — and (b) the decision-critical path free of any LLM dependency, consistent with §3.5’s rule that the LLM must not modify calculations or override decision policy. `test_e2e_explanation_never_alters_decision_score` confirms the exact `decision_score` number appears unmodified in the generated text.

9. TESTS EXECUTED / PASSED / FAILED

37/37 passed (36 original + 1 added during the post-delivery audit – see Section 6B). Coverage: query parsing (int, sentence, unparseable), Data Agent found/not-found, Market Agent’s reuse of Phase 8’s exact market-strength formula (byte-for-byte equality check, not just “similar”), Prediction Agent never calling REJECTED models (checked via both the limitation text and the absence of DOM/sale-probability keys in output), Evidence Agent citation-ID integrity, the citation cross-contamination regression test across 5 properties, Finance Agent’s Missing Input Policy (property 1 → INSUFFICIENT DATA, confirmed with explicit assumptions → PASS), Risk Agent’s permanent UNCALIBRATED status and confidence cap, Geo Agent’s flood-risk INSUFFICIENT DATA passthrough, Decision Policy weight-sum/renormalization/gate-failure/threshold logic (including the high-score-low-confidence-becomes-HOLD case), and 8 full end-to-end tests covering all four decision states plus contract completeness, explanation fidelity, and reproducibility.

10. MASTER PROMPT REQUIREMENTS COVERAGE

Section	Requirement	Status
9.2	11 agents (Query Understanding through Explanation)	✓ all built
9.3	Standard AgentFinding contract, no uncontrolled prose	✓ <code>contracts.py</code>

9.4	Deterministic DecisionPolicy, explicit/configurable/documented/testable weights	✓ <code>decision_policy.py</code>
9.5	INVEST/HOLD/AVOID/INSUFFICIENT states	✓ all 4 confirmed reachable
9.6	Decision score vs. decision confidence kept separate	✓ enforced structurally, risk cap tested
9.7	Critical gates before INVEST	✓ <code>check_critical_gates</code> , gate-failure test
9.8	Final DecisionResult, all 19 fields	✓ <code>contracts.DecisionResult</code> , field-completeness test
9.9	End-to-end flow	✓ <code>orchestrator.run_decision_pipeline</code>
3.4	Deterministic calculations, not LLM	✓ zero LLM calls anywhere in this phase
3.5	LLM governance (N/A here — no LLM used)	✓ Explanation Agent is a template by design, documented

11. BUSINESS VALUE

- A single function call (`run_decision_pipeline(property_id)`) now produces a fully-reasoned, fully-cited, fully-tested investment decision — the platform’s central promise (\$0, \$22).
- The confidence/score separation prevents the single most common decision-support failure mode: a good-looking number being mistaken for a trustworthy one. Property 2731’s INVEST (score 69.2, confidence 81.7) and any hypothetical high-score-but-thin-data property would be visibly different in this system, not silently equivalent.
- The two bugs found and fixed during this phase’s own build are a live demonstration of exactly the discipline the Master Prompt asks for throughout — build, test against real data, and fix what breaks, documented plainly rather than hidden.

12. KNOWN LIMITATIONS (see also `docs/limitations.md`)

1. Decision score’s linear component mappings (prediction/financial scaling factors) are documented judgement calls, not fitted — no historical outcome dataset exists to fit them against.
2. Decision confidence’s `risk_completeness` is permanently capped at 60/100 — by design, reflecting Phase 7’s permanent UNCALIBRATED status, not a defect.
3. The Explanation Agent is a deterministic template, not the LLM-based explanation layer Phase 10 will add.
4. Market/Prediction/Financial/Risk/Geo weights (\$9.4) are a documented default (20/15/25/20/20%), not fitted to real investment outcomes.
5. Same 44.5% OBSERVED-financial-data coverage limitation inherited from Phase 6/7 — properties outside that coverage correctly return INSUFFICIENT via the Finance Agent’s Missing Input Policy, cascading up through the gates.
6. No live PostgreSQL was available in this sandbox; run with `--source csv` (documented dev/test path).

13. DEPENDENCIES

Phases 4, 5, 6, 7, 8 (all reused unchanged, per Section 3 above). Transitively, Phase 1.

14. SECURITY CONSIDERATIONS

Unchanged pattern from every prior phase: no hardcoded credentials, parameterized SQL where used,

`DB_USER / DB_PASSWORD` required with no defaults. No LLM calls in this phase, so no prompt-injection surface exists here (Phase 5's prompt-injection scanner already guards the evidence corpus this phase's Evidence Agent reads from).

15. PERFORMANCE CONSIDERATIONS

The full 26-property demo batch (every agent + 1,000-simulation Monte Carlo per property) completed in well under a minute in this sandbox. Not measured at full-portfolio scale (8,000 properties); per §15's standing rule, should be measured before any optimization decision.

16. PRODUCTION READINESS

Per Section 24: **Production-Oriented Prototype**. All 11 agents, the deterministic policy, and the end-to-end flow are complete and tested against real data with two real bugs found and fixed during build. Recommended before Phase 10 integration: (a) re-run with `--source db` against a live PostgreSQL instance; (b) have the business review and confirm the Decision Policy's default weights and thresholds (§5 above) before they're used for real investment decisions; (c) confirm whether the platform should surface Model G's (`risk_score_v1_composite`, UNCALIBRATED) output anywhere in Phase 10's dashboards, since it was deliberately excluded from this phase's `decision_score` to avoid double-counting risk against Phase 7's Monte Carlo output.

17. NEXT PHASE DEPENDENCIES

Phase 10 (Decision Experience, Power BI & AI Explanation) should: (a) consume `DecisionResult` objects directly from `orchestrator.run_decision_pipeline()` for its Dashboard 6 (Decision Intelligence); (b) build the actual LLM-based AI Explanation Layer (§10.10) on top of the same `DecisionResult` this phase's template-based Explanation Agent already consumes — the LLM must preserve every numeric value exactly, exactly as `explanation_agent.py`'s docstring already requires of itself; (c) Dashboard 7 (Data Quality & Monitoring) should surface this phase's `component_scores` and `gate_results` directly, since they're already structured for that purpose.

18. FINAL STATUS

PHASE 9: COMPLETE. 11 agents, a deterministic Decision Policy with score/confidence separation and critical gates, and a full end-to-end orchestrator — all built on top of Phases 4-8's engines reused unchanged, 37/37 automated tests passing (including 1 added during a post-delivery audit that caught an inherited Phase 6 ROI bug – see Section 6B), two real integration bugs found and fixed during build, and all four decision states confirmed reachable with real properties. The platform can now answer “*Should I invest in Property X?*” end-to-end. Ready for Phase 10.